PIL

Prompt Intelligence Layer

Product Requirements Document

Version 1.0 | March 2026 | Confidential

Product	Status	Owner	Date
PIL v1 (Core ML)	Pre-seed / Concept	Founding Team	March 2026

1. Executive Summary

PIL (Prompt Intelligence Layer) is a middleware product that intercepts, refines, and optimises prompts before they reach any Large Language Model API. It addresses a fundamental and growing problem in the AI tooling industry: developers, data scientists, and enterprise teams routinely send bloated, poorly-structured prompts to LLMs — wasting tokens, money, and output quality.

Vision	PIL becomes the standard middleware layer between humans and LLMs — the way Nginx is standard between the internet and web servers.

V1 focuses entirely on the core ML intelligence — the models, APIs, compression pipeline, and ethics layer. V2 delivers the VS Code and JetBrains extensions that bring PIL directly into the developer's IDE.

1.1 The Problem

Token misuse is the hidden tax of AI adoption. Research and practitioner surveys consistently show:

- 60–70% of tokens in average developer prompts are redundant, filler, or poorly-structured context
- Developers have no real-time feedback on token consumption as they type
- Multi-turn conversations compound the problem — conversation history is included verbatim, not intelligently compressed
- Enterprise teams have no audit trail of what was sent to AI providers — a compliance and security gap
- PII and API secrets are routinely included in prompts, creating data leakage risk

1.2 The Solution

PIL is a proxy API layer with four core capabilities: intent extraction, token budget management, rolling context compression, and a mandatory ethics/audit rail. Every prompt that passes through PIL is denser, safer, and more likely to produce high-quality output — at 30–60% lower token cost.

2. Goals & Success Metrics

2.1 V1 Goals

- Ship a production-grade PIL API proxy (FastAPI, Python) deployable as SaaS and on-prem
- Achieve measurable 30%+ token reduction on a benchmark suite of 1,000 real developer prompts
- Implement zero-data-retention mode and pass a basic security review
- Onboard 50 design partner developers within 60 days of alpha release
- Reach API latency overhead < 80ms p95 for the compression pipeline

2.2 Success Metrics

Metric	Baseline	V1 Target	V2 Target
Token reduction rate	0%	≥ 30%	≥ 50%
Output quality score (human eval)	—	≥ 4.0 / 5.0	≥ 4.4 / 5.0
API latency added (p95)	—	< 80ms	< 40ms
PII detection recall	—	≥ 97%	≥ 99.5%
MAU (Monthly Active Users)	0	500	10,000
Design partners	0	50	200

3. Target Users

3.1 Primary — V1

Individual Developers (API Users)

Developers making direct calls to OpenAI, Anthropic, or Gemini APIs. They care about cost and output quality. They will integrate PIL as a drop-in proxy endpoint. Acquisition channel: GitHub, HackerNews, dev Twitter.

AI/ML Engineers

Teams building LLM-powered products and pipelines. They care about determinism, cost at scale, and auditability. They will use the PIL SDK or proxy in CI/CD pipelines.

3.2 Secondary — V1 into V2

Engineering Leads & CTOs

Decision-makers approving AI tooling spend. They care about ROI, compliance, and vendor risk. The savings dashboard and audit log are their primary value proposition.

Enterprise Security & Compliance Teams

Gatekeepers for any tool that touches code or internal data. They need zero-retention guarantees, PII scrubbing, and an audit trail. PIL's ethics layer is designed specifically for this persona.

4. Core Features — V1 Scope

4.1 Intent Extraction Engine

A fine-tuned small model (distilBERT or Phi-3-mini) that classifies every prompt into a structured intent schema: task type (generate / debug / refactor / explain / document / test), programming language, framework, output format, and constraints. The classified intent drives downstream prompt template selection.

Tech	distilBERT fine-tuned on 500K labelled developer prompt pairs. Inference via ONNX Runtime. Latency target: < 20ms.

4.2 Token Budget Engine

Accepts a user-defined or system-defined token budget. Scores every clause in the input prompt by information density. Prunes low-density filler. Restructures remaining content into the most token-efficient representation for the target LLM's context window format.

- Tokeniser support: tiktoken (OpenAI), Anthropic tokeniser, SentencePiece (open-source models)
- Priority scoring uses TF-IDF weighted by intent classification
- Budget modes: strict (hard cutoff), adaptive (quality-aware), economy (maximum compression)

4.3 Rolling Context Summariser

In multi-turn conversations, conversation history grows unbounded. PIL maintains a rolling summary of prior turns using a fast summarisation model, replacing verbatim history with a dense digest. The developer's context is preserved; the token count is not.

Tech	facebook/bart-large-cnn fine-tuned on code conversation pairs. Runs asynchronously to avoid blocking the main prompt path.

4.4 Structured Prompt Builder

Assembles the final optimised prompt from: the classified intent, compressed context, retrieved few-shot examples (from a semantic cache), and the target LLM's preferred prompt format. Outputs are schema-validated before dispatch.

4.5 Ethics & Audit Rail

Every prompt passes through this layer before and after PIL processing. It is not optional and cannot be disabled.

- PII detection and redaction using Microsoft Presidio (names, emails, phone numbers, SSNs, credit cards)
- Secret scanning: regex patterns for AWS keys, OpenAI keys, GitHub tokens, SSH private keys
- Bias flag: prompts touching hiring, lending, medical decisions, or criminal justice are routed to a stricter template and flagged in the audit log

- Structured audit log: timestamp, original token count, optimised token count, cost delta, PII entities found, provider used. Log is local-first, never transmitted without explicit opt-in
- Zero-retention mode: no prompt content stored after processing. Enforced at the infrastructure level, not just policy

4.6 Semantic Cache

Prompts are embedded using a lightweight embedding model (all-MiniLM-L6-v2) and compared against a vector store (Qdrant). Similar prompts (cosine similarity > 0.92) return cached responses. This is the compounding moat — the cache improves with every request processed.

5. Out of Scope — V1

V2 The following are explicitly deferred to V2 and the IDE extension work.

- VS Code extension and JetBrains plugin
- GitHub Copilot Chat participant integration
- Real-time inline token meter in editor
- Local on-device classifier (llama.cpp / phi-3-mini offline)
- Git diff context harvesting
- Team collaboration features (shared prompt templates, org-level audit dashboards)
- Fine-tuning on customer-specific codebases

6. Technical Architecture — V1

6.1 Stack Overview

Layer	Technology	Rationale
API Proxy	FastAPI (Python 3.11)	Async, fast, type-safe. Industry standard for ML APIs
Intent Classifier	distilBERT / ONNX Runtime	< 20ms inference, runs on CPU, no GPU required
Summariser	BART-large-cnn (HuggingFace)	Best compression/quality ratio for conversation turns
PII Scrubber	Microsoft Presidio	Production-grade, multi-entity, extensible
Token Counting	tiktoken + Anthropic SDK	Exact counts, not estimates
Vector Store	Qdrant (self-hosted)	Fast, open-source, Rust-based, < 5ms query
Embeddings	all-MiniLM-L6-v2	384-dim, CPU-friendly, strong semantic quality
Queue	Celery + Redis	Async summarisation and cache warm jobs
Observability	Prometheus + Grafana	Standard OSS stack, customer-facing dashboard ready
Deployment	Docker + Kubernetes	Cloud-agnostic. AWS/GCP/Azure and on-prem

6.2 Request Flow

- Client sends prompt to PIL proxy endpoint (drop-in replacement for provider URL)
- PII scrubber runs — secrets and entities replaced with placeholders
- Semantic cache lookup — cache hit returns immediately with no LLM call
- Intent classifier runs — task type, language, framework, output format extracted
- Token budget engine scores and compresses the prompt
- Rolling context summariser updates conversation history digest
- Structured prompt builder assembles final prompt for target provider
- Optimised prompt dispatched to LLM provider (OpenAI / Anthropic / Gemini / OSS)
- Response returned to client with X-PIL-Tokens-Saved and X-PIL-Cost-Delta headers
- Audit log entry written asynchronously

7. Ethics & Data Governance

PIL is built on an ethics-first architecture. These are not policies — they are technical constraints enforced at the infrastructure level.

7.1 Data Minimisation

PIL processes prompts in memory and discards them immediately after optimisation in zero-retention mode. No prompt content is written to disk or transmitted to PIL servers without explicit customer opt-in. The audit log records only metadata (token counts, entity types found, cost delta) — not prompt content.

7.2 Transparency

Every response from PIL includes headers showing exactly what was changed: tokens removed, compression ratio, entities scrubbed, cache hit/miss. Customers can inspect the diff between original and optimised prompts at any time. There are no silent modifications.

7.3 No Training on Customer Data

PIL's models are trained exclusively on consented public datasets and synthetic data. Customer prompts are never used for training, fine-tuning, or model evaluation. This is enforced contractually (enterprise DPA) and architecturally (no training pipeline has access to the production prompt store).

7.4 Consent Architecture

- Free tier: local PIL only, zero network calls for prompt content
- Cloud tier: customer explicitly opts in to cloud compression, sees exactly what is transmitted
- Enterprise tier: fully on-premises, no data leaves the customer's VPC under any circumstances

8. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Compression loses critical context	Medium	High	Quality eval suite; user can review diff and reject any optimisation
PII detection misses edge cases	Low	High	Presidio + custom regex; quarterly red-team reviews; user can add custom entity patterns
LLM provider API changes break integration	Medium	Medium	Abstraction layer; versioned adapters per provider; automated smoke tests
Latency overhead is unacceptable	Low	High	ONNX Runtime for inference; async summariser; aggressive caching; circuit breaker to bypass PIL if latency spikes
Competitors ship similar feature	High	Medium	Speed to market; proprietary semantic cache dataset; ethics layer as differentiator
Enterprise security review blocks adoption	Medium	High	SOC 2 Type II audit in roadmap Q3 2026; penetration test before enterprise GA

9. Roadmap

Phase	Timeline	Deliverables
Alpha (V1)	April – May 2026	PIL proxy API, intent classifier, token budget engine, PII scrubber, audit log. 50 design partners.
Beta (V1)	June – July 2026	Rolling context summariser, semantic cache, savings dashboard, SDK (Python + Node). 200 MAU.
GA (V1)	August 2026	Production hardening, SOC 2 audit begins, enterprise tier, on-prem Helm chart. 1,000 MAU.
V2 Alpha	September 2026	VS Code extension: token meter, inline PII scrubber, intent classifier, savings panel.
V2 Beta	October – November 2026	JetBrains plugin, Copilot Chat participant, git diff context harvesting, local offline mode.
V2 GA	December 2026	Full IDE integration suite, team features, org-level audit dashboard.

Document Classification: Confidential — Internal Use Only
PIL Product Team | Version 1.0 | March 2026